

Ejercicio 1. Método signo

Análisis: El método recibe un entero y devuelve 1 si es positivo, -1 si es negativo y 0 si es cero.

- **Complejidad Ciclomática:** 3 (Dos condiciones if + 1).
- **Casos de prueba:**
 - TC01: Entrada 5, Esperado 1 (Rama $x > 0$).
 - TC02: Entrada -5, Esperado -1 (Rama $x < 0$).
 - TC03: Entrada 0, Esperado 0 (Rama de salida final).

Código JUnit:

```
Java
@Test
void testSigno() {
    assertEquals(1, Ejercicios.signo(10));
    assertEquals(-1, Ejercicios.signo(-10));
    assertEquals(0, Ejercicios.signo(0));
}
```

Ejercicio 2. Método clasificarEdad

Análisis: Clasifica edades en etapas de la vida y lanza una excepción si la edad es menor de 0.

- **Complejidad Ciclomática:** 7 (Seis condiciones de rango + 1).
- **Casos de prueba:** Usamos valores límite como -1, 0, 6, 12, 18, 25 y 60.

Código JUnit:

```
Java
@Test
void testClasificarEdad() {
    assertThrows(IllegalArgumentException.class, () -> Ejercicios.clasificarEdad(-1));
    assertEquals("Infancia", Ejercicios.clasificarEdad(0));
    assertEquals("Niñez", Ejercicios.clasificarEdad(6));
    assertEquals("Adolescencia", Ejercicios.clasificarEdad(12));
    assertEquals("Juventud", Ejercicios.clasificarEdad(18));
    assertEquals("Adulthood", Ejercicios.clasificarEdad(25));
    assertEquals("Vejez", Ejercicios.clasificarEdad(60));
}
```

```
}
```

Ejercicio 3. Método contarPositivos

Análisis: Cuenta cuántos números en un array son mayores que cero.

- **Complejidad Ciclomática:** 3 (El bucle for y el if interno).
- **Casos:** Array vacío, array sin positivos y array con mezcla.

Código JUnit:

```
Java
@Test
void testContarPositivos() {
    assertEquals(0, Ejercicios.contarPositivos(new int[]{}));
    assertEquals(0, Ejercicios.contarPositivos(new int[]{-1, -2, 0}));
    assertEquals(2, Ejercicios.contarPositivos(new int[]{1, -1, 5}));
}
```

Ejercicio 4. Método calificacion

Análisis: Transforma una nota de 0 a 10 en texto usando un switch.

- **Complejidad Ciclomática:** 6 (La validación inicial y los 5 casos del switch).
- **Mejora:** Los casos de "SUSPENSO" (0-4) se podrían agrupar con @ParameterizedTest.

Código JUnit:

```
Java
@Test
void testCalificacion() {
    assertThrows(IllegalArgumentException.class, () -> Ejercicios.calificacion(11));
    assertEquals("SUSPENSO", Ejercicios.calificacion(4));
    assertEquals("SUFICIENTE", Ejercicios.calificacion(5));
    assertEquals("BIEN", Ejercicios.calificacion(6));
    assertEquals("NOTABLE", Ejercicios.calificacion(8));
    assertEquals("SOBRESALIENTE", Ejercicios.calificacion(10));
}
```

Ejercicio 5. Método esBisiesto

Análisis: Implementamos la lógica de años bisiestos (divisible por 4 y no por 100, o divisible por 400).

- **Complejidad Ciclomática:** 4.

Código del método:

```
Java
public static boolean esBisiesto(int anyo) {
    return (anyo % 4 == 0 && anyo % 100 != 0) || (anyo % 400 == 0);
}
```

Ejercicio 6. Piedra - Papel - Tijera

Análisis: Compara dos jugadas. Lanza error si no son "PIEDRA", "PAPEL" o "TIJERA".

Código del método:

```
Java
public static String jugar(String j1, String j2) {
    if (!j1.matches("PIEDRA|PAPEL|TIJERA") || !j2.matches("PIEDRA|PAPEL|TIJERA")) {
        throw new IllegalArgumentException("Jugada no válida");
    }
    if (j1.equals(j2)) return "EMPATE";
    if ((j1.equals("PIEDRA") && j2.equals("TIJERA")) ||
        (j1.equals("TIJERA") && j2.equals("PAPEL")) ||
        (j1.equals("PAPEL") && j2.equals("PIEDRA"))) {
        return "GANA JUGADOR 1";
    }
    return "GANA JUGADOR 2";
}
```

Ejercicio 7. Validador de contraseñas

Análisis: Suma puntos según la seguridad de la clave y devuelve un nivel.

Código del método:

Java

```
public static String evaluarPassword(String password) {  
    int puntos = 0;  
    if (password.length() >= 8) puntos++;  
    if (password.matches(".*[A-Z].*")) puntos++;  
    if (password.matches(".*[a-z].*")) puntos++;  
    if (password.matches(".*[0-9].*")) puntos++;  
    if (password.matches(".*[!@#$%^&*.]*")) puntos++;  
  
    if (puntos <= 2) return "DEBIL";  
    if (puntos <= 4) return "MEDIA";  
    return "FUERTE";  
}
```

Test JUnit:

Java

@Test

```
void testPassword() {  
    assertEquals("DEBIL", Ejercicios.evaluarPassword("123"));  
    assertEquals("FUERTE", Ejercicios.evaluarPassword("ClaveSegura1!"));  
}
```